



PRESIDENCY UNIVERSITY

Private University Estd. in Karnataka State by Act No. 41 of 2013

Itgalpura, Rajankunte, Yelahanka| Bengaluru – 560064



School of Artificial Intelligence and Advanced Computing (PSAIAC)

Department of Cyber Security and Intelligent Systems

B.Tech Computer Science and Technology Devops (CDV)

V SEMESTER / III YEAR

SOFTWARE TESTING LAB MANUAL

COURSE NAME	SOFTWARE TESTING LAB
COURSE CODE	CDV 2504
COURSE CREDIT STRUCTURE	0-0-2-1
COURSE IC	Mr.JAI KUMAR B
PREPARED BY	Mr.JAI KUMAR B

List of Tools:

1. Manual Testing Tools

- TestLink – Test management and documentation
- Bugzilla – Bug reporting and tracking
- MantisBT – Lightweight issue tracker

2. Unit Testing Tools

- JUnit – Unit testing for Java
- TestNG – Advanced unit testing for Java
- PyTest – Unit testing for Python
- NUnit – Unit testing for .NET applications

3. Automation Testing Tools

- Selenium IDE – Record-and-playback for browser testing
- Selenium WebDriver – Code-based automation for web apps
- Katalon Recorder – Selenium-based test recorder (Chrome plugin)

4. Mobile Application Testing

- Appium – Automation for Android and iOS apps

5. Integration / CI Tools

- Jenkins – Continuous integration, pipeline integration
- GitHub Actions – Cloud-based CI/CD automation

6. Performance and Load Testing Tools

- Apache JMeter – Load, stress, and performance testing for web applications
- Gatling – High-performance load testing

7. Code Coverage Tools

- JaCoCo – Java code coverage
- Coverage.py – Python code coverage
- Clover – Code coverage for Java/Groovy

8. Code Quality & Static Analysis

- SonarQube – Code quality and security analysis
- PMD – Java source code analyzer
- FindBugs / SpotBugs – Bug detection in Java

9. Cross-Browser Testing

- Selenium Grid – Parallel testing on different browsers/machines
- Browsershots – Web design testing in various browsers

TEXTBOOKS:

T1: Fundamentals of Software Testing, 2nd Edition (Revised & Updated) *Bernard Homès* Released: July 2024

T2: Software Testing, 5th Edition *Geoff Thompson et al.* Publication: March 2024

T3: Software Testing Strategies: A Testing Guide for the 2020s *Matthew Heusser & Michael Larsen*, Publication: December 2023

REFERENCES:

R1: A Comprehensive Study of Mutation Testing Techniques Jia, Yue; Harman, Mark IEEE Transactions on Software Engineering

R2: Software Testing in the Era of Artificial Intelligence: Survey, Landscape and Future Directions Ahmed, S.; Ali, N.; Mirza, H.T. 2020

R3: Test Automation in Continuous Integration: A Systematic Literature Review Garousi, Vahid;2017 Felderer, Michael
R4: Boris Beizer, Software Testing Techniques, Dreamtech Press, 2nd Edition, 2003.

WEB RESOURCES:

W1: <https://cryptography.io/> – Python cryptographic library and resources.
W2: <https://nvlpubs.nist.gov/> – NIST publications on cryptographic standards.
W3: <https://www.owasp.org/> – Open Web Application Security Project for best practices.
W4: <https://www.tutorialspoint.com/cryptography/index.htm> – Cryptography basics and tutorials.
W5: <https://www.coursera.org/learn/crypto> – Stanford's free online cryptography course on Coursera

LABSHEET 1: Introduction to software testing, SDLC vs STLC, writing simple test cases

Introduction to Software Testing

Software testing is the process of evaluating and verifying that a software application or system meets the specified requirements and is free of defects. It ensures the quality, functionality, security, and performance of the software.

Types of Testing:

- Manual Testing
- Automation Testing
- Functional Testing
- Non-functional Testing

Goals:

- Detect bugs and errors.
- Ensure the product meets user expectations.
- Validate that the system works as intended.

SDLC (Software Development Life Cycle)

SDLC is the end-to-end process of software development that includes:

1. Requirement Analysis
2. Design
3. Implementation (Coding)
4. Testing
5. Deployment
6. Maintenance

STLC (Software Testing Life Cycle)

STLC is the testing-focused life cycle, part of the SDLC that includes:

1. Requirement Analysis (for testability)
2. Test Planning
3. Test Case Design
4. Test Environment Setup
5. Test Execution
6. Test Cycle Closure

Writing Simple Test Cases

A test case is a document that describes an input, action, or event and the expected result, to determine if a feature works as intended.

Test Case ID	Description	Input	Expected Output	Status
TC001	Add two numbers	2 + 3	5	Pass
TC002	Login with invalid	user,123	Show error msg	Pass

Write at least two test cases for a login functionality where a user provides a username and password. Consider both valid and invalid inputs.

Test Case ID	Description	Input	Expected Output
TC001	Valid login credentials	Username: admin Password: admin123	Redirect to dashboard/home page
TC002	Invalid login (wrong password)	Username: admin Password: wrongpass	Show error message "Invalid login"

1. Question:

Password Strength Validation

Q: Design test cases for a password strength checker that should validate:

- Minimum 8 characters
- At least one uppercase letter
- At least one number
- At least one special character

LABSHEET 2: Manual testing – Functional and Non-functional test case design for a sample application.

What is Manual Testing?

Manual testing is the process of manually executing test cases without the use of automation tools, to ensure the application behaves as expected.

Functional Testing

Functional testing checks if the system functions according to the requirements. It focuses on:

- What the system does
- Testing user interactions, APIs, databases, UI, etc.

Example Functional Test Areas:

- Login/logout
- Form submission
- User registration
- File upload

Non-Functional Testing

Non-functional testing checks how well the system performs rather than what it does. It includes:

- Performance
- Security
- Usability
- Compatibility

Example Non-Functional Test Areas:

- How fast does the page load?
- Is it secure against unauthorized access?
- Is the interface user-friendly?
- Does it work across all browsers?

Sample Application: Login Page

Functional Test Cases:

Test Case ID	Description	Input	Expected Output
TC01	Valid login	Username: user, Password: 1234	Login successful, redirect to home
TC02	Invalid password	Username: user, Password: abcd	Error: Incorrect credentials
TC03	Blank username	Username: "", Password: 1234	Error: "Username is required"
TC04	Blank fields	Both fields empty	Error: "Enter username and password"
TC05	Remember me checkbox	Checked	Login credentials retained on next visit

Non-Functional Test Cases:

Test Case ID	Description	Criteria/Test Type	Expected Result
NFC01	Login response time	Performance	Login should complete within 2 sec
NFC02	UI elements are readable	Usability	Font size $\geq$ 12px, labels clear
NFC03	Password field masked	Security	Characters appear as bullets/stars
NFC04	Cross-browser compatibility	Compatibility	Login works on Chrome, Firefox, Edge
NFC05	Handle 50 concurrent logins	Load test (Performance)	System should not crash or hang

2. Question:

You are given a login page of a web application that contains:

- Username field
- Password field
- Login button
- "Remember Me" checkbox
- Forgot Password link

1. Write 5 functional test cases for the login feature.

2. Write 3 non-functional test cases for the same login feature.

****3. Clearly mention the input, expected output, and type of testing for each test case.**

LABSHEET 3: Boundary Value Analysis (BVA) and Equivalence Class Partitioning

1. Boundary Value Analysis (BVA)

```
#include <stdio.h>

int main() {
    int a;
    printf("Enter side of triangle (1-20): ");
    scanf("%d", &a);

    if (a >= 1 && a <= 20)
        printf("Valid input\n");
    else
        printf("Invalid input\n");

    return 0;
}
```

Scenario: Triangle side length must be between 1 and 20

Test Case	Input (a)	Expected Output	Description
TC1	0	Invalid input	Just below minimum
TC2	1	Valid input	Minimum value
TC3	2	Valid input	Just above minimum
TC4	19	Valid input	Just below maximum
TC5	20	Valid input	Maximum value
TC6	21	Invalid input	Just above maximum

2. Equivalence Class Partitioning (ECP)

Scenario: Same triangle side validation (1 to 20)

Class Type	Test Case	Input	Expected Output	Description
Valid class	TC1	10	Valid input	Within valid range
Invalid class (low)	TC2	0	Invalid input	Less than min
Invalid class (high)	TC3	25	Invalid input	Greater than max

LABSHEET 4: Decision Table and State Transition testing techniques

1. Decision Table Testing

```
#include <stdio.h>
#include <string.h>

int main() {
    char username[20], password[20];

    printf("Enter username: ");
    scanf("%s", username);

    printf("Enter password: ");
    scanf("%s", password);

    if (strcmp(username, "") == 0 && strcmp(password, "") == 0)
        printf("Error: Missing Credentials\n");
    else if (strcmp(username, "admin") != 0)
        printf("Error: Invalid Username\n");
    else if (strcmp(password, "admin123") != 0)
        printf("Error: Invalid Password\n");
    else
        printf("Login successful\n");

    return 0;
}
```

Test Case	Input Username	Input Password	Output
TC1	admin	admin123	Login successful
TC2	user	admin123	Error: Invalid Username
TC3	admin	pass	Error: Invalid Password
TC4			Error: Missing Credentials

2. State Transition Testing

```
#include <stdio.h>
#include <string.h>

int main() {
    char correctPIN[] = "1234";
    char inputPIN[10];
    int attempts = 0;
    int state = 0; // 0 = Card Inserted, 1 = Transaction Allowed, 2 = Blocked

    while (attempts < 3 && state == 0) {
        printf("Enter PIN: ");
```

```

scanf("%s", inputPIN);

if (strcmp(inputPIN, correctPIN) == 0) {
    state = 1;
    break;
} else {
    attempts++;
    printf("Incorrect PIN. Attempt %d of 3.\n", attempts);
}
}

if (state == 1)
    printf("Transaction Allowed.\n");
else
    printf("Card Blocked due to 3 wrong attempts.\n");

return 0;
}

```

OUTPUT 1

Enter PIN: 1234
Transaction Allowed.

OUTPUT 2

Enter PIN: 1111
Incorrect PIN. Attempt 1 of 3.
Enter PIN: 2222
Incorrect PIN. Attempt 2 of 3.
Enter PIN: 3333
Incorrect PIN. Attempt 3 of 3.
Card Blocked due to 3 wrong attempts.

Test case Table:

TC	Current State	Input	Next State	Output / Message
TC1	S0 (Start)	Correct PIN	S1	Transaction Allowed
TC2	S0	Incorrect PIN	S0 (1st attempt)	Incorrect PIN. Attempt 1 of 3.
TC3	S0	Incorrect PIN	S0 (2nd attempt)	Incorrect PIN. Attempt 2 of 3.
TC4	S0	Incorrect PIN (3rd)	S2	Card Blocked due to 3 wrong attempts.
TC5	S2	Any Input	S2	No further attempts – card remains blocked

LABSHEET 5: Path coverage and control flow graph-based white-box testing

Program: White-box testing (Path Coverage)

```
#include <stdio.h>

int main() {
    int a, b, max;

    printf("Enter two numbers: ");
    scanf("%d %d", &a, &b);

    if (a > b)
        max = a;
    else
        max = b;

    printf("Maximum is: %d\n", max);
    return 0;
}
```

Control Flow Graph (CFG)

The control flow of the above program can be broken into these nodes:

```
[1] Start →
[2] Read a, b →
[3] if (a > b) →
    └─ True → [4] max = a
    └─ False → [5] max = b →
[6] Print max →
[7] End
```

Paths in the CFG:

Path	Input (a, b)	Expected Output	Explanation
Path 1	10, 5	Maximum is: 10	$a > b \rightarrow \text{max} = a$
Path 2	3, 9	Maximum is: 9	$a \leq b \rightarrow \text{max} = b$

Independent Paths for Path Coverage

Path	Input (a, b)	Expected Output	Explanation
Path 1	10, 5	Maximum is: 10	$a > b \rightarrow \text{max} = a$
Path 2	3, 9	Maximum is: 9	$a \leq b \rightarrow \text{max} = b$

NB: Goal of Path Coverage:

- Cover all independent paths in the control flow.
- Ensure both conditions of the if statement are tested.

1.Program

StudentResult.java

```
public class StudentResult {

    /**
     * Computes the student's result string based on marks and attendance.
     *
     * Decision logic:
     * 1) invalid marks? -> "Invalid"
     * 2) marks < 40 -> "Fail"
     * 3) 40 <= marks < 60 -> depends on attendance:
     *    - attended -> "Pass"
     *    - not attended -> "Fail (attendance)"
     * 4) 60 <= marks < 80 -> "Merit"
     * 5) marks >= 80 -> "Distinction"
     */
    public static String computeResult(int marks, boolean attended) {
        // Decision 1: invalid input
        if (marks < 0 || marks > 100) {
            return "Invalid";
        }

        // Decision 2: fail if < 40
        if (marks < 40) {
            return "Fail";
        }

        // Decision 3: pass region 40-59 (attendance matters)
        if (marks < 60) {
            if (attended) {
                return "Pass";
            } else {
                return "Fail (attendance)";
            }
        }

        // Decision 4: merit 60-79
        if (marks < 80) {
            return "Merit";
        }

        // Decision 5: >= 80
        return "Distinction";
    }

    // simple manual-run demo
    public static void main(String[] args) {
        int[] testMarks = {-1, 30, 55, 55, 70, 85};
        boolean[] testAttendance = {true, true, true, false, true, true};

        for (int i = 0; i < testMarks.length; i++) {
```

```

        System.out.printf("marks=%d attended=%b -> %s%n", testMarks[i], testAttendance[i],
            computeResult(testMarks[i], testAttendance[i]));
    }
}
}

```

OUTPUT:

```

marks=-1    attended=true -> Invalid
marks=30    attended=true -> Fail
marks=55    attended=true -> Pass
marks=55    attended=false -> Fail (attendance)
marks=70    attended=true -> Merit
marks=85    attended=true -> Distinction

```

2) Control Flow Graph (CFG) — description & nodes

I'll describe nodes and edges; you can draw this as a standard flow chart.

Nodes:

1. Start / entry
2. Check marks < 0 || marks > 100 (Decision D1)
3. Return "Invalid"
4. Check marks < 40 (Decision D2)
5. Return "Fail"
6. Check marks < 60 (Decision D3)
7. Check attended (Decision D4)
8. Return "Pass"
9. Return "Fail (attendance)"
10. Check marks < 80 (Decision D5)
11. Return "Merit"
12. Return "Distinction"
13. End

Edges (major ones):

- 1 → 2
- 2 (true) → 3
- 2 (false) → 4
- 4 (true) → 5
- 4 (false) → 6
- 6 (true) → 7
- 7 (true) → 8
- 7 (false) → 9
- 6 (false) → 10
- 10 (true) → 11
- 10 (false) → 12

(Each return edge ends at End node.)

If you draw it: decisions are diamond nodes; normal flow continues down; nested decision for attendance is under the 40–59 branch.

3) Cyclomatic complexity (V(G)) — computed step-by-step (digit-by-digit arithmetic)

Method has 5 decision points:

- D1: marks < 0 || marks > 100
- D2: marks < 40

- D3: marks < 60
- D4: attended
- D5: marks < 80

Using the simple formula: $V(G) = \text{number_of_decisions} + 1$

Number_of_decisions = 5

So $V(G) = 5 + 1 = 6$

That means you need at least 6 independent paths (test cases) to achieve basis path coverage.

4) Independent paths and test inputs (one test per independent path)

Below are the 6 independent paths (with concrete input values you can use). These are chosen to exercise each decision outcome.

1. Invalid marks — hits D1 true $\Rightarrow$ returns "Invalid"
Input: marks = -1, attended = true
Expected: "Invalid"
2. Fail (marks < 40) — D1 false, D2 true $\Rightarrow$ returns "Fail"
Input: marks = 30, attended = true
Expected: "Fail"
3. Pass (40–59, attended) — D1 false, D2 false, D3 true, D4 true $\Rightarrow$ "Pass"
Input: marks = 55, attended = true
Expected: "Pass"
4. Fail due to attendance (40–59, not attended) — D3 true, D4 false $\Rightarrow$ "Fail (attendance)"
Input: marks = 55, attended = false
Expected: "Fail (attendance)"
5. Merit (60–79) — D3 false, D5 true $\Rightarrow$ "Merit"
Input: marks = 70, attended = true
Expected: "Merit"
6. Distinction (≥ 80) — D5 false $\Rightarrow$ "Distinction"
Input: marks = 85, attended = true
Expected: "Distinction"

Those six tests correspond exactly to the cyclomatic complexity lower bound.

Test cases:

Path	Input	Expected Output	Remarks
1	marks = -1, attended = true	Invalid	Pass
2	marks = 30, attended = true	Fail	Pass
3	marks = 55, attended = true	Pass	Pass
4	marks = 55, attended = false	Fail (attendance)	Pass
5	marks = 70, attended = true	Merit	Pass
6	marks = 85, attended = true	Distinction	Pass

LABSHEET 6: JUnit testing for Java programs – Unit Testing

Program Under Test: Calculator.java

```
public class Calculator {
    public int add(int a, int b) {
        return a + b;
    }

    public int subtract(int a, int b) {
        return a - b;
    }

    public int multiply(int a, int b) {
        return a * b;
    }

    public int divide(int a, int b) {
        if (b == 0) throw new IllegalArgumentException("Division by zero");
        return a / b;
    }
}
```

JUnit Test File: CalculatorTest.java

```
import org.junit.Test;
import static org.junit.Assert.*;

public class CalculatorTest {
    Calculator calc = new Calculator();

    @Test
    public void testAdd() {
        assertEquals(10, calc.add(6, 4));
    }

    @Test
    public void testSubtract() {
        assertEquals(2, calc.subtract(6, 4));
    }

    @Test
    public void testMultiply() {
        assertEquals(24, calc.multiply(6, 4));
    }

    @Test
    public void testDivide() {
        assertEquals(2, calc.divide(8, 4));
    }
}
```

```
@Test(expected = IllegalArgumentException.class)
public void testDivideByZero() {
    calc.divide(8, 0);
}
}
```

Expected Output (from JUnit in IDE or terminal)

- ✓ testAdd() passed
- ✓ testSubtract() passed
- ✓ testMultiply() passed
- ✓ testDivide() passed
- ✓ testDivideByZero() passed (expected exception)

Requirements:

- JDK installed
- JUnit 4 or 5 library included
- Run with IDE (Eclipse/IntelliJ) or via command line using javac and java

Program Under Test: Calculator.java

```
public class Calculator {

    // Method to add two numbers
    public int add(int a, int b) {
        return a + b;
    }

    // Method to subtract two numbers
    public int subtract(int a, int b) {
```

```

        return a - b;
    }

    // Method to multiply two numbers
    public int multiply(int a, int b) {
        return a * b;
    }

    // Method to divide two numbers
    public int divide(int a, int b) {
        if (b == 0) {
            throw new IllegalArgumentException("Division by zero");
        }
        return a / b;
    }

    // Main method to test the Calculator
    public static void main(String[] args) {
        Calculator calc = new Calculator();

        System.out.println("Addition: " + calc.add(10, 5));
        System.out.println("Subtraction: " + calc.subtract(10, 5));
        System.out.println("Multiplication: " + calc.multiply(10, 5));
        System.out.println("Division: " + calc.divide(10, 5));
    }
}

```

JUnit Test File: CalculatorTest.java

```

import static org.junit.Assert.*;
import org.junit.Test;

public class CalculatorTest {

    Calculator calc = new Calculator();

    @Test
    public void testAdd() {
        assertEquals(15, calc.add(10, 5));
        assertEquals(0, calc.add(-5, 5));
    }

    @Test
    public void testSubtract() {
        assertEquals(5, calc.subtract(10, 5));
        assertEquals(-10, calc.subtract(-5, 5));
    }

    @Test
    public void testMultiply() {
        assertEquals(50, calc.multiply(10, 5));
        assertEquals(-25, calc.multiply(-5, 5));
    }
}

```

```
}

@Test
public void testDivide() {
    assertEquals(2, calc.divide(10, 5));
    assertEquals(-2, calc.divide(-10, 5));
}

@Test(expected = IllegalArgumentException.class)
public void testDivideByZero() {
    calc.divide(10, 0);
}
}
```

LABSHEET 7: Writing automation test scripts using Selenium IDE

Objective:

- Understand how to record, play, and manage test cases using Selenium IDE (a Chrome/Firefox extension).
- Create a simple automation script without writing code manually.

Test Scenario: Login Form Automation

Sample Web Application:

You can use any dummy login page like:

URL: <https://practicetestautomation.com/practice-test-login/>

Steps in Selenium IDE:

Install Selenium IDE Extension:

- Chrome/Firefox > Extensions > Search "Selenium IDE" > Add to browser.
- Open Selenium IDE:
- Click the Selenium icon in the toolbar.
- Create a New Project:
- Click "Create a new project" → Name: LoginTest.
- Start Recording:
- Selenium IDE will open a browser window.
- Go to <https://practicetestautomation.com/practice-test-login/>
- Perform Actions:
- Enter username: student
- Enter password: Password123
- Click Login button.
- Stop Recording.

TEST SCRIPT IN SELENIUM IDE:

Command	Target	Value
open	/practice-test-login/	
type	id=username	student
type	id=password	Password123
click	css=button[type='submit']	
assertText	css=h1	Logged In Successfully

Expected Output:

- Selenium IDE successfully runs the steps.
 - After clicking "Run current test case", you should see
- ✓ Test case passed: LoginTest

Test Case Summary:

Step	Action	Input	Expected Result
1	Open URL	/login	Login page loads
2	Type Username	student	Username entered
3	Type Password	Password123	Password entered
4	Click Login	-	Redirects to success page
5	Assert Text	"Logged In..."	Validation passes

LABSHEET 8:Developing automated test cases using Selenium WebDriver

Developing Automated Test Cases using Selenium WebDriver

Objective:

- Use Selenium WebDriver to automate the login functionality of a sample web application.

Test Scenario:

Automate the login process for this sample site:

URL: <https://practicetestautomation.com/practice-test-login/>

Credentials:

- Username: student
- Password: Password123

Program: Selenium WebDriver Automation

```
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.chrome.ChromeDriver;

public class LoginAutomation {
    public static void main(String[] args) {

        // Set the path to chromedriver executable
        System.setProperty("webdriver.chrome.driver", "C:\\path\\to\\chromedriver.exe");

        // Initialize Chrome WebDriver
        WebDriver driver = new ChromeDriver();

        // Open the login page
        driver.get("https://practicetestautomation.com/practice-test-login/");

        // Maximize the browser
        driver.manage().window().maximize();

        // Locate and fill username
        WebElement username = driver.findElement(By.id("username"));
        username.sendKeys("student");

        // Locate and fill password
        WebElement password = driver.findElement(By.id("password"));
        password.sendKeys("Password123");

        // Click the login button
        WebElement loginButton = driver.findElement(By.id("submit"));
        loginButton.click();

        // Verification step: check if login was successful
        WebElement message = driver.findElement(By.tagName("h1"));

        // Output the result
        if (message.getText().equals("Logged In Successfully")) {
```

```
        System.out.println("Test Passed: Login Successful");
    } else {
        System.out.println("Test Failed: Login Not Successful");
    }

    // Close browser
    driver.quit();
}
}
```

Expected Output (in console):

Test Passed: Login Successful

If credentials are wrong:

Test Failed: Login Not Successful

Setup Instructions:

1. Download and place the ChromeDriver in your system.
2. Set the correct path in `System.setProperty("webdriver.chrome.driver", "C:\\path\\to\\chromedriver.exe");`
3. Add **Selenium JARs** to your project (selenium-java library).
4. Run using any Java IDE (like Eclipse/IntelliJ).

Selenium JARs

<https://www.selenium.dev/downloads/>

LABSHEET 9: Cross-browser testing using Selenium Grid

Cross-Browser Testing using Selenium Grid

Objective:

- Use Selenium WebDriver to automate the login functionality of a sample web application.

Test Scenario:

Automate the login process for this sample site:

URL: <https://practicetestautomation.com/practice-test-login/>

Credentials:

- Username: student
- Password: Password123

Program: Selenium WebDriver Automation

```
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.chrome.ChromeDriver;

public class LoginAutomation {
    public static void main(String[] args) {

        // Set the path to chromedriver executable
        System.setProperty("webdriver.chrome.driver", "C:\\path\\to\\chromedriver.exe");

        // Initialize Chrome WebDriver
        WebDriver driver = new ChromeDriver();

        // Open the login page
        driver.get("https://practicetestautomation.com/practice-test-login/");

        // Maximize the browser
        driver.manage().window().maximize();

        // Locate and fill username
        WebElement username = driver.findElement(By.id("username"));
        username.sendKeys("student");

        // Locate and fill password
        WebElement password = driver.findElement(By.id("password"));
        password.sendKeys("Password123");

        // Click the login button
        WebElement loginButton = driver.findElement(By.id("submit"));
        loginButton.click();

        // Verification step: check if login was successful
        WebElement message = driver.findElement(By.tagName("h1"));

        // Output the result
        if (message.getText().equals("Logged In Successfully")) {
```

```
        System.out.println("Test Passed: Login Successful");
    } else {
        System.out.println("Test Failed: Login Not Successful");
    }

    // Close browser
    driver.quit();
}
}
```

Expected Output (in console):

Test Passed: Login Successful

If credentials are wrong:

Test Failed: Login Not Successful

Setup Instructions:

1. Download and place the ChromeDriver in your system.
2. Set the correct path in `System.setProperty("webdriver.chrome.driver", "C:\\path\\to\\chromedriver.exe");`
3. Add Selenium JARs to your project (selenium-java library).
4. Run using any Java IDE (like Eclipse/IntelliJ).

LABSHEET 10: Integration of Selenium with JUnit/TestNG for automation testing

Integration of Selenium with TestNG for Automation Testing

Objective:

- Integrate Selenium with TestNG.
- Automate a login test case using assertions.
- Generate structured test reports.

Test Scenario:

- Visit: <https://practicetestautomation.com/practice-test-login/>
- Enter valid username/password.
- Click login.
- Verify the success message on the next page.

Prerequisites:

1. Tools Required:

- Java JDK (17+)
- Eclipse or IntelliJ
- Maven (for dependency management)
- Selenium WebDriver
- Junit

2. Maven Dependencies (pom.xml):

Create Maven Project

In Eclipse or IntelliJ:

1. Go to **File** → **New** → **Maven Project**.
2. Select **“Create a simple project”** and click **Next**.
3. Enter:
 - Group ID: com.example
 - Artifact ID: selenium-java
4. Finish to create the project.

```
<dependency>
  <groupId>org.seleniumhq.selenium</groupId>
  <artifactId>selenium-java</artifactId>
  <version>4.25.0</version>
</dependency>
```

```
<dependency>
  <groupId>io.github.bonigarcia</groupId>
  <artifactId>webdrivermanager</artifactId>
  <version>5.9.1</version>
</dependency>
```

Then right-click your project → **Maven** → **Update Project**

PROGRAM

```
package org.seleniumhq.selenium.selenium_java;

import org.junit.Assert;
import org.junit.After;
import org.junit.Before;
import org.junit.Test;

import io.github.bonigarcia.wdm.WebDriverManager;

import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.firefox.FirefoxDriver;

public class AppTest {

    WebDriver driver;

    @Before
    public void setUp() {
        // Setup FirefoxDriver automatically using WebDriverManager
        WebDriverManager.firefoxdriver().setup();
        driver = new FirefoxDriver();
        driver.manage().window().maximize();
        driver.get("https://practicetestautomation.com/practice-test-login/");
    }

    @Test
    public void testValidLogin() {
        // Enter login credentials
        driver.findElement(By.id("username")).sendKeys("student");
        driver.findElement(By.id("password")).sendKeys("Password123");
        driver.findElement(By.id("submit")).click();

        // Validate heading
        WebElement heading = driver.findElement(By.tagName("h1"));
        String actualHeading = heading.getText();
        Assert.assertEquals("Login heading did not match", "Logged In Successfully", actualHeading);
        System.out.println("Login heading matched. Test Passed.");

        // Validate URL
        String currentURL = driver.getCurrentUrl();
        Assert.assertTrue("URL validation failed", currentURL.contains("logged-in-successfully"));
        System.out.println("URL validation passed. Test Passed.");
    }

    @After
    public void tearDown() {
        if (driver != null) {
            driver.quit();
        }
    }
}
```

```
}  
}  
}
```

Run the Test

Option 1: From IDE

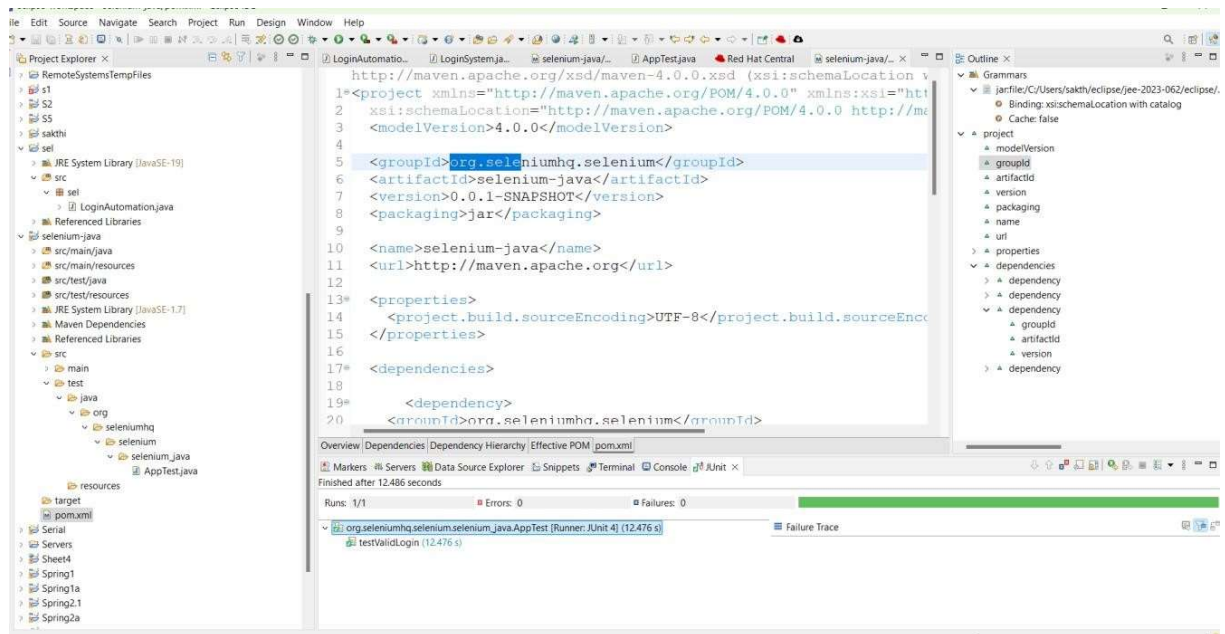
- Right-click on the project file → **Run As** → **Junit Test**

Expected Output (Console):

Junit:

Default test

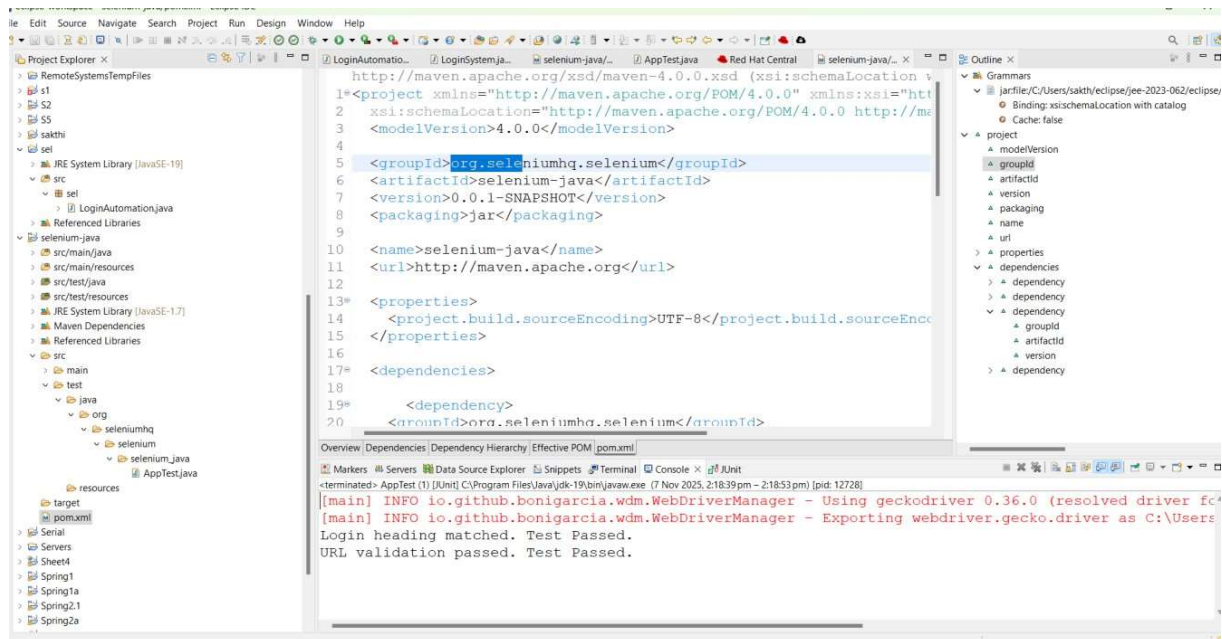
Tests run: 1, Failures: 0, Skips: 0



Eclipse console

Login heading matched. Test Passed.

URL validation passed. Test Passed.



LABSHEET :11 : Bug reporting and tracking using Bugzilla or Mantis

Option 1: Bugzilla (using REST API)

Install Requirements:

pip install requests

Python Script: Report and Track Bug in Bugzilla

```
import requests
```

```
# Configuration
```

```
BUGZILLA_URL = 'https://your-bugzilla-url.com/rest'
```

```
API_KEY = 'your_api_key'
```

```
# Create a new bug
```

```
create_payload = {
```

```
    'product': 'TestProduct',
```

```
    'component': 'TestComponent',
```

```
    'summary': 'Test Bug via API',
```

```
    'version': 'unspecified',
```

```
    'description': 'Steps to reproduce: ...',
```

```
    'op_sys': 'All',
```

```
    'platform': 'All',
```

```
    'api_key': API_KEY
```

```
}
```

```
create_response = requests.post(f'{BUGZILLA_URL}/bug', data=create_payload)
```

```
print("Bug Created:", create_response.json())
```

```
# Track (get) the bug details
```

```
bug_id = create_response.json().get("id")
```

```
track_response = requests.get(f'{BUGZILLA_URL}/bug/{bug_id}?api_key={API_KEY}')
```

```
print("Bug Details:", track_response.json())
```

Sample Output:

Bug Created: {'id': 12345}

Bug Details: {

```
    "bugs": [{
```

```
        "id": 12345,
```

```
        "summary": "Test Bug via API",
```

```
        "status": "NEW",
```

```
        "product": "TestProduct"
```

```
    }]
```

```
}
```

Option 2: MantisBT (using REST API)

Install Requirements:

pip install requests

Python Script: Report and Track Bug in MantisBT

```
import requests
import json

# Configuration
MANTIS_URL = "https://your-mantis-url.com/api/rest"
API_TOKEN = "your_api_token"

headers = {
    "Authorization": API_TOKEN,
    "Content-Type": "application/json"
}

# Create a new issue
create_payload = {
    "summary": "Bug via API",
    "description": "Steps to reproduce...",
    "project": {"name": "TestProject"},
    "category": {"name": "General"}
}
create_response = requests.post(f'{MANTIS_URL}/issues/', headers=headers,
                                json=create_payload)
print("Bug Created:", create_response.json())

# Track the issue
issue_id = create_response.json()['issue']['id']
track_response = requests.get(f'{MANTIS_URL}/issues/{issue_id}', headers=headers)
print("Bug Details:", track_response.json())
```

Sample Output:

```
Bug Created: {"issue": {"id": 101}}
Bug Details: {
  "issue": {
    "id": 101,
    "summary": "Bug via API",
    "status": {"name": "new"},
    "project": {"name": "TestProject"}
  }
}
```

LABSHEET 12:Test case execution and defect tracking on sample project

Scenario: Sample Project – Login Page Testing

Test Case Execution Steps

- Prepare test cases: valid and invalid login credentials.
- Execute each test case.
- Record pass/fail based on expected vs actual results.

Defect Tracking Trigger

- If actual result $\neq$ expected $\rightarrow$ log a defect (manually or using tool like Bugzilla/Mantis).

PSEUDOCODE:

START

```
Define TestCases = [  
  {"id": 1, "username": "admin", "password": "admin123", "expected": "Login Successful"},  
  {"id": 2, "username": "admin", "password": "wrongpass", "expected": "Login Failed"}  
]
```

For each test in TestCases:

 Execute login(username, password)
 Capture actual_result

 If actual_result == expected:

 Mark test case as PASSED

 Else:

 Mark test case as FAILED

 Log defect in Bugzilla/Mantis with test ID, steps, expected vs actual

END

PYTHON CODE SIMULATION (Mock Execution)

```
def login(username, password):  
    # Simulating login logic  
    if username == "admin" and password == "admin123":  
        return "Login Successful"  
    else:  
        return "Login Failed"  
  
# Sample test cases  
test_cases = [  
    {"id": 1, "username": "admin", "password": "admin123", "expected": "Login Successful"},  
    {"id": 2, "username": "admin", "password": "wrongpass", "expected": "Login Successful"} #  
    Intentional fail  
]  
  
# Execute test cases  
for test in test_cases:  
    actual = login(test["username"], test["password"])  
    print(f"Test Case ID: {test['id']}")  
    print(f"Expected: {test['expected']}, Actual: {actual}")  
  
    if actual == test["expected"]:  
        print("Result: ■ PASSED\n")
```

```
else:
    print("Result: + FAILED")
    print("→ Log Defect: Mismatch in login result\n")
```

SAMPLE OUTPUT

Test Case ID: 1
Expected: Login Successful, Actual: Login Successful
Result:  PASSED

Test Case ID: 2
Expected: Login Successful, Actual: Login Failed
Result: + FAILED
→ Log Defect: Mismatch in login result

DEFECT REPORT SAMPLE (Manual Entry Format)

Field	Value
Title	Login fails for valid credentials
Test Case ID	TC_002
Steps	Enter username: admin, password: wrongpass, click Login
Expected	Login Successful
Actual	Login Failed
Priority	High
Status	New

Scenario: Test Web App Homepage Load

1. What is Performance Testing in JMeter?

Performance testing checks:

- **Response time**
- **Throughput**
- **Concurrency**
- **Resource usage**

Pseudocode for Performance Testing using JMeter

START

1. Initialize JMeter Test Plan
2. Configure Thread Group:
 - Number of Users (Threads): 10
 - Ramp-Up Time: 10 seconds
 - Loop Count: 2
3. Add HTTP Request Sampler:
 - Server Name: www.example.com
 - Path: /
4. Add Listener:
 - View Results Tree
 - Summary Report
5. Run Test Plan
6. Collect Metrics:
 - Response Time (Avg, Min, Max)
 - Error %
 - Throughput (requests/sec)

END

Steps in JMeter UI (no code):

Open JMeter

- Add → Test Plan → Name it: *Homepage Load Test*
- Add → Thread Group
- Threads: 10
- Ramp-up: 10
- Loop: 2
- Add → Sampler → HTTP Request
- Server: www.example.com
- Path: /
- Method: GET
- Add → Listener → Summary Report / View Results Tree
- Click  to Run Test

Sample Output from Summary Report

Label	Samples	Avg	Min	Max	Error %	Throughput	KB/sec
/	20	210	180	300	0.00%	1.9/sec	25.3

Explanation of Output Fields

- **Samples:** Total number of requests (10 users $\times$ 2 loops = 20)
- **Avg:** Average response time (ms)
- **Error %:** % of failed requests (e.g., 0.00% = all passed)
- **Throughput:** Requests per second
- **KB/sec:** Data received per second

Optional Pseudocode Extension – Using JMeter CLI

START

1. Create test plan: TestPlan.jmx in JMeter GUI

2. Run it via terminal:

```
jmeter -n -t TestPlan.jmx -l result.jtl -e -o /report
```

3. Analyze:

- Open /report/index.html in browser
- Review graphs: Response time, errors, throughput

END

LABSHEET :14 Preparing test plans, test suites, and test summary reports

Test Plan

Key Points:

1. A test plan outlines the scope, approach, resources, and schedule of testing.
2. It defines what to test, how to test, who will test, and when.
3. Includes risk analysis, tools, entry/exit criteria.

Pseudocode:

START TEST PLAN

1. Identify Test Objectives
2. Define Scope (In-Scope & Out-of-Scope)
3. List Features to be Tested
4. Identify Test Deliverables
5. Define Entry and Exit Criteria
6. Assign Roles and Responsibilities
7. Plan Schedule and Milestones
8. Identify Risks and Mitigations

END TEST PLAN

Sample Snippet:

Test Plan: Login Module

Objective: Verify login functionality with valid and invalid credentials

Scope:

In-Scope: Web login form

Out-of-Scope: Forgot Password

Tools: Selenium, JUnit

Entry Criteria: Dev complete & test environment ready

Exit Criteria: All major test cases passed, no critical defects

Schedule: 5–10 July 2025

Test Suite

Key Points:

1. A test suite is a collection of related test cases grouped by module or feature.
2. It helps manage and execute tests efficiently.
3. Often maintained in tools like TestRail, Zephyr, or Excel.

Pseudocode:

START TEST SUITE: Login Module

Include Test Cases:

- TC001 - Valid Login
- TC002 - Invalid Username
- TC003 - Invalid Password
- TC004 - Empty Fields
- TC005 - SQL Injection Test

Link to Test Plan: Login Functional Test Plan

END TEST SUITE

Sample Format:

TC ID	Description	Input Data	Expected Result	Status
TC001	Login with valid data	admin/admin123	Login successful	Pass
TC002	Wrong username	wrong/admin123	Show error	Pass
TC005	SQL Injection	' OR '1'='1	Reject login attempt	Fail

Test Summary Report

Key Points:

1. Summarizes testing activity after completion.
2. Covers test metrics, defects, pass/fail rates, and observations.
3. Helps stakeholders decide on release readiness.

Pseudocode:

START TEST SUMMARY REPORT

1. Project Name, Module, Test Duration
2. Total Test Cases Executed
3. Pass/Fail Statistics
4. List of Open/Critical Defects
5. Observations & Recommendations
6. Exit Criteria Met?
7. Sign-Off Decision

END TEST SUMMARY REPORT

Sample Summary Report:

Test Summary Report – Login Module

Module Tested: Login

Test Period: 5–8 July 2025

Total Test Cases: 10

Passed: 9

Failed: 1

Defects: 1 critical defect (SQL Injection not blocked)

Observations: System is functional, but vulnerable to SQL injection.

Recommendation: Fix critical defect before production deployment.

Sign-Off: + Not Ready